UNIVERSITY OF MALAYA

WIX1002 : FUNDAMENTALS OF PROGRAMMING

VIVA 1: FLOW CONTROL

SEM 1 2025 / 2026

LECTURER NAME:

DR. NURUL BINTI JAPAR

GROUP MEMBERS:

1. JASMINE CHIN JIA YEE (25006180)
2. JOSEPHINE DING JIE YU (25005876)
3. NG YUE QHI (25005961)
4. NG SHAO ERN (25006089)
5. JASMINE CHIN YING HUI (25005998)
6. NG GEOK LIU (25005946)

Question 1: The Tok Wan's Number Charms and the Pasar Malam Challenge

Section 1 : Problem

Tok Wan's have a special ability to create numerous "charms". He will take 3 numbers : Initial value **a**, Multiplier **b**, Charm Length, **n**. A group of young people approached him with a series of number inquiries, **q**. For each inquiry, they will present Tok Wan with the 3 numbers, Initial value **a**, Multiplier **b**, Charm Length, **n**. My goal is to solve the sequence of numbers that form Tok Wan's charm.

Section 2 : Solution

My first approach to this question is there will be two loops required. First loop **i**, is to continue the calculation process for number of inquiries, **q** , while the second loop **j**, is to calculate each number for charm length, **n**. In the second loop, calculation is made based on the equation given in the question a + (b * 2 ^ j). After calculation, the answer will be printed. To move the next query's answer to the next line, a print is needed after each second loop **j**, is finished performed.

Besides, based on question requirements I will need to prepare a while loop for each number's input. This step is to ensure that the value that user input is within the range required by the question.

Section 3 : Sample Input & Output

```
Enter number of inquiries : 501
Enter number of inquiries :
```

Users will need to reenter the number of inquiries if the previous value entered exceeds 500.

```
Enter initial value : -1
Enter initial value :
```

Users will need to reenter the initial value if the previous value entered is negative.

```
Enter multiplier seed : 52
Enter multiplier seed :
```

Users will need to reenter the multiplier seed if the previous value entered is greater than 50.

```
Enter charm length : 16
Enter charm length : 0
Enter charm length :
```

Users will need to reenter the charm length if the previous value is 0 or greater than 15.

```
Enter number of inquiries : 2
Enter initial value : 0
Enter multiplier seed : 2
Enter charm length : 10
2 4 8 16 32 64 128 256 512 1024
Enter initial value : 5
Enter multiplier seed : 3
Enter charm length : 5
8 11 17 29 53
```

When the user enters all the numbers in the correct range, calculation will be carried out based on charm length **n**, and number of inquiries **q**.

Section 4 : Java Source Code

```
import java.util.*;
public class Question1 {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        int q = 0;
        int a = 51;
        int b = 51;
        int n = 0;
        while (q > 500 || q < 1){
            System.out.print("Enter number of inquiries : ");
            q = input.nextInt();
        }
        for (int i = 0; i < q; i++){
            do{
                System.out.print("Enter initial value : ");
                a = input.nextInt();
            }while(a > 50 || a < 0);

            do{
                System.out.print("Enter multiplier seed : ");
                b = input.nextInt();
            }while(b > 50 || b < 0);

            do{
                System.out.print("Enter charm length : ");
                n = input.nextInt();
            }while(n > 15 || n <= 0);

            for (int j = 0; j < n; j++){
                int digit = (int) (a + (b * Math.pow(2, j)));
                System.out.print(digit + " ");
            }
            System.out.println();
        }
    }
}
```

Question 2 : Ah Hock's Digital Signature

Section 1 : Problem

This problem focuses on determining the “Digital Signature” of a given number based on Ah Hock’s unique digit-analysis method. For each inquiry, a number **N** and a chosen **Lucky Digit L** are provided. The number is examined digit by digit, and each digit is classified into one of four categories: **lucky digits** that match L, **zeros** (unless L = 0, in which case zeros are counted as lucky), **even digits** (2, 4, 6, 8), and **odd digits** (1, 3, 5, 7, 9). A special rule applies when N = 0, where it is treated as a single-digit “0”. After counting all categories, the system determines the number’s **Digital Signature** using strict priority rules: the result is **LUCKY** if the lucky digit count is strictly the highest, **BALANCED** if the even digit count is highest, **ENERGETIC** if the odd digit count is highest, and **NEUTRAL** for any ties or when the zero count dominates.

Section 2 : Solution Explanation

Firstly, system accepts the number of inquiries, T between 1 to 200. For each test case, it then reads two values: number N and a Lucky Digit L that is from 0 to 9. Four counters are declared and initialized to zero: luckyCount, zeroCount, evenCount, and oddCount. These will store the number of digits that fall into each respective category.

For special case N=0, it is treated as a single digit. If L=0, then the digit is counted as a Lucky Digit. Otherwise, it is counted as a Zero Digit. If N is not zero, a while loop is used to process each digit. The last digit is obtained using digit = N % 10. Then, N is reduced using integer division N = N / 10. If the digit equals L, luckyCount increases by 1. Else if the digit is 0 and L is not equals to 0, increment zeroCount. Else if the digit is even, increment evenCount. Otherwise, increment oddCount.

After all digits are analyzed, comparisons are made among the four counters. If luckyCount is strictly greater than all others, “LUCKY” is stored in the array message. Else if evenCount is strictly greater than all others, it stores “BALANCED”. Else if oddCount is strictly greater than all others, “ENERGETIC” is stored. Otherwise, the array will store “NEUTRAL”. Lastly, system prints out the message stored in array message[i] according to each N entered for T times.

Section 3 : Sample input and output

Test 1

Input

```
5
40628 4
500 0
909090 9
123456789 2
0 0
```

Output

```
BALANCED
LUCKY
NEUTRAL
ENERGETIC
LUCKY
```

Test 2

Input

```
9
0 5
103020 0
1004008 5
7777 7
284682 9
13579 8
1234 6
5151 5
1999999999 9
```

Output

```
NEUTRAL
LUCKY
NEUTRAL
LUCKY
BALANCED
ENERGETIC
NEUTRAL
NEUTRAL
LUCKY
```

Section 4 : Java Source Code

```
import java.util.Scanner;

public class Viva01Q2 {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        int T;
        while (true) {
            T = sc.nextInt();
            if (T >= 1 && T <= 200) {
                break;
            }
            System.out.println("Invalid! Please enter a number between 1 and 200!");
        }

        String []message=new String[T];

        for (int i=0;i<T;i++){
            int N = sc.nextInt();
            int L;

            while(true){
                L=sc.nextInt();
                if(L>=0 && L<10){
                    break;
                }
                System.out.println("Please enter a lucky digit between 0 to 9!");
            }
            int zeroCount=0;
            int luckyCount=0;
            int evenCount=0;
            int oddCount=0;


             if (N==0){
                 if (L==0)
                     message[i]="LUCKY";
                 else
                     message[i]="NEUTRAL";
             }
             else{
             while (N>0){
                 int digit=N%10;
                 N=N/10;

                 if (L==0){
                     zeroCount=0;
                     if(digit==0)
                         luckyCount++;
                     else if (digit%2==0){
                         evenCount++;
                     }
                     else
                         oddCount++;
                 }
```

```
                else if (digit==L){
                    luckyCount++;
                }
                else if (digit==0){
                    zeroCount++;
                }
                else if (digit%2==0){
                    evenCount++;
                }
                else{
                    oddCount++;
                }
            }
            if (luckyCount>evenCount && luckyCount>oddCount && luckyCount>zeroCount)
                message[i]="LUCKY";
            else if (evenCount>oddCount && evenCount>luckyCount && evenCount>zeroCount)
                message[i]="BALANCED";
            else if (oddCount>luckyCount && oddCount>evenCount && oddCount>zeroCount)
                message[i]="ENERGETIC";
            else
                message[i]="NEUTRAL";
            }

        }

        System.out.println(" ");

        for (int i=0;i<T;i++){
            System.out.println(message[i]);
        }

    }
}
```

Question 3: Puan Norah's Digital Kolam

Section 1: Problem

Puan Norah is famous for her beautiful kolam designs which she generates using a program. Her program can create geometric patterns of different sizes and styles. Students are tasked with replicating this program by reading two main inputs for each pattern: the Height (H), which determines the pattern's size, and the Style (S), a single character (‘A’ or ‘P’), which determines the pattern's structure.

The program must be able to generate two main styles:

A	Angled	A right-angled number staircase where the number for each row i is the row number itself, repeated i times.
P	Pyramid	A symmetrical pyramid. Row i is a palindrome of numbers counting up from 1 to i and then back down to 1. The entire pattern must be centered using blank spaces.

Section 2: Solution

The number of inquiries, **T,** is read first. A **do-while loop** prompts the user to enter the number of queries **(T**) until the input is between **1 and 50** (inclusive). A **for** loop iterates **T** times. The second **do-while loop** forces the user to re-enter input if:

1. The height is **not** from 1-9, **OR**
2. The style is **not** 'A' and **not** 'P'.

Then, the Height (an integer) and Style (a character) are read for the current test case. A simple **if-else if** structure directs control flow to the appropriate pattern printing required by the user.

A. Style 'A'

This pattern is a simple number staircase. I use nested loops: The outer loop controls the number of rows (from **h**=1 to height). The inner loop controls how many times the number is printed in

each row (from **i**=1 to **h**). The value of the number printed is the row number. After printing each row, it moves to a new line using System.out.println().

B. Style 'P'

This pattern requires three sequential nested loops per row **h.** The first loop prints **blank spaces** for centering. For a height, the number of spaces needed for row **h** is **height-h**. The second loop prints **ascending numbers** starting from 1 up to **i.** The third loop completes the palindrome by printing **descending numbers** starting from **h - 1** and decreasing down to 1. After the three inner loops complete, System.out.println() is called to move to the next row.

Section 3 : Sample Input & Output

Test 1: Normal Case

```
run:
Enter number of queries: 3
Enter the height and style: 4 A
1
22
333
4444
Enter the height and style: 4 P
   1
  121
 12321
1234321
Enter the height and style: 6 P
     1
    121
   12321
  1234321
 123454321
12345654321
BUILD SUCCESSFUL (total time: 34 seconds)
```

Test 2: If user enter number of queries larger than 50 or smaller than 1

```
Enter number of queries(1-50): 51
Enter number of queries(1-50): 0
Enter number of queries(1-50):
```

Test 3: If user enter the height larger than 9 or smaller than 1

```
Enter number of queries(1-50): 4
Enter the height(1-9) and style(A/P): 0 A
Enter the height(1-9) and style(A/P): 10 A
Enter the height(1-9) and style(A/P): 4 A
1
22
333
4444
Enter the height(1-9) and style(A/P):
```

Test 4: if user enter the character other than ‘A’ & ‘P’

```
Enter the height(1-9) and style(A/P): 9 Y
Enter the height(1-9) and style(A/P): 9 R
Enter the height(1-9) and style(A/P):
```

Section 4: Source Code

```
public static void main(String[] args) {
    Scanner sc=new Scanner(System.in);
    int T=0;
    int height=0;
    char style;

    do {
        System.out.print("Enter number of queries(1-50): ");
        T = sc.nextInt();
    }while (T<1 || T>50);

    for (int t=1 ; t<=T ; t++){
        do {
            System.out.print("Enter the height(1-9) and style(A/P): ");
            height = sc.nextInt();
            style = sc.next().charAt(0);
        }while ((height<1 || height>9) || (style!='A' && style!='P'));

        if (style=='A'){
            for (int h = 1; h <= height; h++) {
                for (int i= 1; i <= h; i++) {
                    System.out.print(h);
                }
                System.out.println(); // move to next line
             }

        } else if (style=='P'){
            for (int h=1; h<=height ; h++) {
                for (int i=1 ; i<=height-h ; i++){ //Printing spaces
                    System.out.print(" ");
                }
                for (int j=1 ; j<=h ; j++){ //Printing ascending number
                    System.out.print(j);
                }
                for (int k=h-1; k>0 ; k-- ){ //Printing descending number
                    System.out.print(k);
                }
                System.out.println(); //move to next line
            }
        }
    }
}
```

Question 4: The Tale of Tok Wan Osman, Word Gems, and Hidden Values at the Pasar Malam

Section 1 : Problem

The task is to analyse a given word and a gem length k, and identify three special substrings ("gems") of that length:

1. "First Whisper": Lexicographically smallest substring (appear first in a dictionary).
2. "Last Echo": Lexicographically largest substring (appear last in a dictionary).
3. "Core Value": Substring with the highest total sum of ASCIl values of its letters (If two substring have the same total sum, the first substring encountered is chosen).

The analysis must be conducted on the word after converting it to lowercase. The final output must list the three gems on separate lines in the order: First Whisper, Last Echo and Core Value.

Section 2 : Solution

First, the program prompts the user to input a word and a gem length k.

- The word is validated to ensure it contains only English letters and does not exceed 50 characters. The user will be prompted to re-enter if input is invalid.
- The word is converted to lowercase to make all comparisons case-insensitive.
- The gem length k is validated to ensure it is positive and does not exceed the length of the word. The user will be prompted to re-enter if input is invalid.

Next, the first k characters of the word are manually combined using a loop to initialise the three substrings: First Whisper, Last Echo and Core Value.

The program then loops through all possible starting positions for substrings of length k:

- Each substring is manually built using a nested loop.
- Compare the substring lexicographically with current smallest and largest substring to update “First Whisper” and “Last Echo”.
- Calculate the sum of ASCII values for the substring. Update “Core Value” if the sum is greater. If multiple substrings have the same sum, the first one encountered is chosen.

Finally, the program prints the three results in order: “First Whisper”, “Last Echo” and “Core Value”.

Section 3 : Sample Input & Output

Test 1

```
Enter a word: satayisverysedap
Enter gem length (length of substring): 3
First Whisper: ata
Last Echo: yse
Core Value: rys
BUILD SUCCESSFUL (total time: 7 seconds)
```

Tests normal lowercase input with no special cases.

Test 2

```
Enter a word: SatayIsVerySedap
Enter gem length (length of substring): 3
First Whisper: ata
Last Echo: yse
Core Value: rys
BUILD SUCCESSFUL (total time: 9 seconds)
```

Tests uppercase input handling. Even though the input contains uppercase letters, the output should be identical to Test Case 1 because the program automatically converts everything to lowercase.

Test 3

```
Enter a word: applebanana
Enter gem length (length of substring): 4
First Whisper: anan
Last Echo: pple
Core Value: pple
BUILD SUCCESSFUL (total time: 3 seconds)
```

Tests a different gem length (k = 4) to ensure the program handles various substring sizes correctly.

Test 4

```
Enter a word: abzaby
Enter gem length (length of substring): 3
First Whisper: aby
Last Echo: zab
Core Value: abz
BUILD SUCCESSFUL (total time: 19 seconds)
```

Tests the tie-breaking rule for ASCII sums. "abz", "bza", and "zab" have the same ASCII total (317), but only the first ("abz") is selected for Core Value.

Test 5

```
Enter a word: thisisalongspecialwordthatalreadyexceedsthewordlimit
The word must not be longer than 50 characters. Please try again.
Enter a word:
```

Prompts the user to re-enter the word if the word is longer than 50 characters.

Test 6

```
Enter a word: @username
The word must contain only English alphabetic letters (A–Z or a–z). Please try again.
Enter a word:
```

Prompts the user to re-enter the word if the word not only contain English alphabetic letters.

Test 7

```
Enter a word: apple
Enter gem length (length of substring): -1
Gem length must be greater than 0. Please try again.
Enter gem length (length of substring): 6
Gem length cannot be greater than the word length. Please try again.
Enter gem length (length of substring):
```

Prompts the user to re-enter the gem length k if k is smaller or equal to zero or k exceeds the length of the word.

Section 4 : Source Code

```java
import java.util.Scanner;

public class Q4 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String word;

        // Input word and validate
        while (true) {
            System.out.print("Enter a word: ");
            word = sc.next();

            // check the word length
            if (word.length() > 50) {
                System.out.println("The word must not be longer than 50 characters. Please try again.");
                continue;
            }

            // check each character is a letter
            boolean validLetters = true;
            for (int i = 0; i < word.length(); i++) {
                char c = word.charAt(i);
                if (!((c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z'))) {
                    validLetters = false;
                    break;
                }
            }

            if (!validLetters) {
                System.out.println("The word must contain only English alphabetic letters (A–Z or a–z). Please try again.");
            } else {
                break;
            }

        }

        word = word.toLowerCase();
```

```java
        // input k and validate
        int k;
        while (true) {
            System.out.print("Enter gem length (length of substring): ");
            k = sc.nextInt();

            if (k <= 0) {
                System.out.println("Gem length must be greater than 0. Please try again.");
            } else if (k > word.length()) {
                System.out.println("Gem length cannot be greater than the word length. Please try again.");
            } else {
                break;
            }
        }

        // initialise firstWhisper manually using loop
        String firstWhisper = "";
        for (int i = 0; i < k; i++) {
            firstWhisper += word.charAt(i);
        }
        String lastEcho = firstWhisper;
        String coreValue = firstWhisper;
        int maxAsciiSum = -1;

        // loop through all possible starting points
        for (int i = 0; i <= word.length() - k; i++) {

            // build substring manually using inner loop
            String substring = "";
            for (int j = i; j < i + k; j++) {
                substring += word.charAt(j);
            }

            // compare for First Whisper
            if (substring.compareTo(firstWhisper) < 0) {
                firstWhisper = substring;
            }

            // compare for Last Echo
            if (substring.compareTo(lastEcho) > 0) {
                lastEcho = substring;
            }

            // calculate ASCII sum manually
            int sum = 0;
            for (int j = 0; j < k; j++) {
                sum += (int) substring.charAt(j);
            }

            // update Core Value if larger sum is found
            if (sum > maxAsciiSum) {
                maxAsciiSum = sum;
                coreValue = substring;
            }
        }

        // Print the output
        System.out.println("First Whisper: " + firstWhisper);
        System.out.println("Last Echo: " + lastEcho);
        System.out.println("Core Value: " + coreValue);
    }
}
```

Question 5 : Uncle Lim's Golden Harmony Lanterns

Section 1 : Problem

The problem asks us to create a program that validates a list of words based on "Uncle Lim's Rule of Golden Harmony". The program first reads an integer, T, representing the number of words to test. For each of the **T** words, it must determine if the word possesses "**Harmony**" or is "**Chaos**".

A word is considered "Chaos" if it breaks at least one of the following two rules:

1. A vowel **cannot be the last letter of the word**.
2. **Two vowels cannot be consecutive to each other.**

For this problem, vowels are defined as 'a', 'e', 'i', 'o', 'u', and all other letters are consonants. If a word does not break any of the rules, it has "Harmony".

Section 2 : Solution

To solve this problem, the program reads several words from the user and determines whether each one follows Uncle Lim's Golden Harmony rules. The program first asks the user for the number of words to test and uses a validation loop to ensure the value is between 1 and 100 before continuing.

For each word, the input is converted to lowercase, and its length is checked to ensure it is between 1 and 50 characters. This ensures that only valid words are processed.

The program then uses the **isChaos()** method to check the two harmony rules. First, it checks whether the last letter of the word is a vowel; if it is, the word is immediately classified as **"Chaos."** Next, the program checks whether the word contains two consecutive vowels. If such a pair exists, the word is also labeled as **"Chaos."**

If the word does not break either rule, it is classified as **"Harmony."** The program prints the result and repeats the process for all remaining words.

Section 3 : Sample Input & Output

Test 1:

```
Enter number of words to test: 4
Word 1: syukur
Result: Harmony

Word 2: meriah
Result: Chaos

Word 3: gembira
Result: Chaos

Word 4: suka
Result: Chaos

------------------------------------------------------
BUILD SUCCESS
```

Test 2:

- Testing for inputs to check if they exceed the range given or not. If the input exceeds the range given, user will be asked to enter again.
- Testing for compressed string with capital letters. The .toLowerCase() method converts the user's input word into all lowercase letters before checking the rules.

```
Enter number of words to test: 101
Number of words must be between 1 and 100, please try again.
Enter number of words to test: 3
Word 1: abcdefghijklmnopqrstuvwxyzabcdefghijklmnopqrstuvwxyz
Length of compressed string must be between 1 and 50. Please try again.
Word 1: LOOP
Result: Chaos

Word 2: Return
Result: Harmony

Word 3: bOOlean
Result: Chaos

------------------------------------------------------------------------
BUILD SUCCESS
```

Section 4 : Source code

```java
import java.util.Scanner;
public class GoldenHarmony_WIX1002_viva1 {
    public static void main(String[] args) {
        Scanner sc = new Scanner (System.in);

        int T;
        // Ask for number of words, which must be between 1 and 100
        while(true){
            System.out.print("Enter number of words to test: ");
            T = sc.nextInt();
            if (T>=1 && T<=100)
                break;
            else
                System.out.println("Number of words must be between 1 and 100, please try again. ");
        }

        // Loop to process each word
        for (int i=1; i<=T; i++){
            String word;
            while (true){
                System.out.print("Word " + i + ": ");
                // Convert input to lowercase to handle uppercase vowels properly
                word = sc.next().toLowerCase();
                if(word.length()>=1 && word.length()<=50)
                    break;
                else
                    System.out.println("Length of compressed string must be between 1 and 50. Please try again. ");
            }

            System.out.print("Result: ");
            if (isChaos(word))
                System.out.println("Chaos \n");
            else
                System.out.println("Harmony \n");
        }
    }


    // Method to check if a character is a vowel
    public static boolean isVowel (char v){
        return v == 'a' || v == 'e' || v == 'i' || v == 'o' || v == 'u';
    }

    // Method to check if a word breaks the rule
    public static boolean isChaos(String word){

        // Rule 1: Last letter cannot be a vowel
        char lastLetter = word.charAt(word.length() - 1);
        if (isVowel(lastLetter)){
            return true; // Chaos
        }

        // Rule 2: No two consecutive vowels allowed
        for (int i=0; i<word.length()-1; i++){
            char currentLetter = word.charAt(i);
            char nextLetter = word.charAt(i+1);

            if (isVowel(currentLetter) && isVowel(nextLetter)){
                return true; //Chaos
            }
        }
        // If no rules are broken → Harmony
        return false;
    }
}
```

Question 6: Alex's Stutter Decompression

Section 1: Problem

This problem asks us to write a decompressor for Alex. The program will read a compressed log file and, if it's valid, calculate the length of the fully decompressed message. However, the format is very fragile, and many logs are corrupted. Hence, the program must also detect some invalid logs.

Decompression and Validation Rules:

- Letters: If you read a lowercase letter, it is simply appended to the decompressed message.
- Digits ('2'-'9'): If you read a digit d from '2' to '9', it is a "stutter" command. Look at the character immediately before the digit in the original compressed string. Then, append that character to the decompressed message d-1 additional times.

The process must stop immediately if:

- the very first character of the compressed string is a digit
- a digit is preceded by another digit
- the log contains the digits '0' or '1'

Section 2: Solution

To solve this, we have to have two functions, one is the main function and one is the decompress function to process the compressed log.

In the main function, we will accept the number of compressed log (T) and compressed log as inputs and assign it as the parameter for the decompress function. Then, the decompress function will be called . We will also check the conditions for tips provided in the question which is:

- Number of compressed log must be between 1 and 100
- Compressed log should only consist of lowercase English letters and digit 0-9
- Length of compressed string must be between 1 and 50
- Length of final decompressed string must not exceed 200 characters

In the decompress function, we will declare an empty String for our decompressed message. Next, we have 4 conditions to be checked when looping through the input

- if the first index is digit: Return “invalid log” where invalid=true
- if digit 1/0 exists: Return “invalid log” where invalid=true
- if character is letter: Append the decompressed message with the letter
- if character is digit:

  Return “invalid log” if previous character is also a digit

  OR

  Append the decompressed message with the previous character for (digit-1) times

Finally, print “invalid log” for invalid cases or the length of the decompressed message if the compressed log is valid.

Section 3: Sample Input&Output

```
run:
Enter number of compressed log:
5
Enter compressed log 1
a4b2
6
Enter compressed log 2
log5
7
Enter compressed log 3
4bidden
Invalid Log
Enter compressed log 4
test1ng
Invalid Log
Enter compressed log 5
xy22z
Invalid Log
BUILD SUCCESSFUL (total time: 29 seconds)
```

Section 4: Source Code

```java
package vival;
import java.util.Scanner;

public class Vival {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter number of compressed log: ");
        int T = sc.nextInt();

        // Tips 1: Validate number of logs (must be between 1 and 100)
        while (T < 1 || T > 100) {
            System.out.println("Number of logs should be between 1 and 100. Please enter again: ");
            T = sc.nextInt();}

        // Loop for each compressed log input
        for (int i = 0; i < T; i++) {
            System.out.println("Enter compressed log " + (i + 1));
            String log = sc.next();

            // Validate each character in the log
            for (int j = 0; j < log.length(); j++) {
                // Tips 2: Check for uppercase letters
                if (Character.isUpperCase(log.charAt(j))) {
                    System.out.println("The compressed log string will only contain lowercase English letters. Please enter again: ");
                    log = sc.next(); }

                // Tips 3: Check string length
                else if (log.length() < 1 || log.length() > 50) {
                    System.out.println("The length of the compressed string will be between 1 and 50 characters. Please enter again: ");
                    log = sc.next();}}
            // Call method
            decompress(log);
        }
    }
```

```java
public static void decompress(String log) {
    boolean invalid = false;    // Mark invalid logs
    String message = "";        // Store decompressed message
    // Loop through each character of the compressed string
    for (int i = 0; i < log.length(); i++) {
        char ch = log.charAt(i);
        // Case 1: Character is a letter
        if (Character.isLetter(ch)) {
            ch = Character.toLowerCase(ch);
            message += ch;}
        // Case 2: Character is a digit
        else if (Character.isDigit(ch)) {
            // Digits 0 and 1 are invalid
            if (ch == '0' || ch == '1') {
                invalid = true;
                break;}
            // A digit cannot appear continuously
            else if (i == 0 || Character.isDigit(log.charAt(i - 1))) {
                invalid = true;
                break;}
            else {
                int numRepeat = (ch - '0') - 1; //repeat letter in front
                for (int j = 0; j < numRepeat; j++) {
                    message += log.charAt(i - 1); }}}
        // Case 3: Any other symbol=invalid
        else {
            invalid = true;
            break;}}
    // Final validation before printing
    if (invalid || message.length() > 200) {
        System.out.println("Invalid Log");}
        else {
        // Tips 4: Final decompressed string must be <= 200 characters
        System.out.println(message.length());}}}
```